AMITY INSTITUTE OF INFORMATION TECHNOLOGY

Course Code : CSE2052

“Introduction to AIML”

<u>LABORATORY RECORD</u>

Fall Semester - 2025

<u>SUBMITTED TO:</u>

Dr Madhu Bhan

<u>SUBMITTED BY:</u>

Name: Madiha mufassira

SEN: A86904824018

BCA

		INDEX	
Sl. No.	**Date**	**Program**	**Page No.**
1	**24/09/25**	Simple Rule-Based Agent	
2	**24/09/25**	Simple Rule-Based Agent	
3	**08/10/25**	Problem representation	
4	**08/10/25**	Simple Rule-Based Agent	
5	**15/10/25**	Goal based agent	
6	**29/10/25**	Breadth First Search	
7	**12/11/25**	Depth First Search	
8	**19/11/25**	Propositional Logic	
9	**19/11/25**	Predicate Facts	
10	**26/11/25**	Mean, Mode, Median and Standard Deviation	
11	**26/11/25**	Dataset	
12	**03/12/25**	Simple Imputer	
13	**10/12/25**	Z-Score	
14	**10/12/25**	Train test	
15	**24/12/25**	Normalization	
16	**31/12/25**	Simple Linear Regression	
17	**07/01/26**	The best-fit regression line	
18	**21/01/26**	Decision tree classifier	
19	**21/01/26**	Logistic Regression	
20	**28/01/26**	K-means	

Program 1:Simple rule based agent.

Write a Python program where agent decides an action based on conditions.

```
weather=input("Enter the weather(sunny,rainy,cloudy,cold):").lower()
if weather=="sunny":
    print("Wear sunglasses")
elif weather=="rainy":
    print("Carry an umbrella")
elif weather=="cloudy":
    print("It might rain,better take a light jacket")
elif weather == "cold":
    print("Wear a warm coat")
else:
    print("No specific advice, enjoy your day!")
```

OUTPUT:

```
Enter the weather(sunny,rainy,cloudy,cold): chgjh
No specific advice, enjoy your day!
```

Program 2 :Simple rule based agent.

Write a Python program where agent decides an action based on Temperature.

```
temperature=input("Enter the temperature(10,20,30,40):").lower()
if temperature=="10":
    print("Low temperature")
elif temperature=="20":
    print("Modem temperature")
elif temperature=="30":
    print("High temperature")
elif temperature== "40":
    print("Too High temperature")
else:
    print("Temperature not detected")
```

OUTPUT:

```
Enter the temperature(10,20,30,40): 30
High temperature
```

Program 3:Problem Representation.

Write a python program 8_puzzle ,8_puzzle state representation. Display as a 3x3 matrix.

```
initial_state = [1, 2, 3, 4, 5 ,6, 7, 8, -1]
goal_state = [2, 4, 6, 8, 14, 16, 10, 12, 1]
print("Initial State:")
for i in range(0, 9, 3):
    print(initial_state[i:i+3])
print("\nGoal State:")
for i in range(0, 9, 3):
    print(goal_state[i:i+3])
if initial_state == goal_state:
    print("The two states are same")
else:
   print("The two states are different")
```

OUTPUT:

```
Initial State:
[1, 2, 3]
[4, 5, 6]
[7, 8, -1]

Goal State:
[2, 4, 6]
[8, 14, 16]
[10, 12, 1]
The two states are different
```

Program 4:Simple rule based agent.

Write a Python program to simulate a Vacuum cleaner

Agent that cleans two locations.

```
def vacuum_agent():
    environment = {}
    environment['A'] = input("Current status of Room A (Dirty/Clean): ").lower()
    environment['B'] = input("Current status of Room B (Dirty/Clean): ").lower()
    vacuum_location = input("Enter initial location of vacuum cleaner (A/B): ").upper()
    print("\nInitial Environment State:")
    print(f"Room A: {environment['A']}")
    print(f"Room B: {environment['B']}")
    print(f"Vacuum Cleaner at Room: {vacuum_location}")
 if environment[vacuum_location] == 'dirty':
        print(f"\nRoom {vacuum_location} is dirty. Cleaning")
        environment[vacuum_location] =='clean'
        print(f"Room{vacuum_location} is cleaned.")
    else:
        print(f"\nRoom {vacuum_location} is already clean. Move to next Room.")
 next_room = 'B' if vacuum_location == 'A'  else 'A'
    if environment[next_room] == 'dirty':
        print(f"\nRoom {next_room} is dirty. Cleaning")
    else:
        print(f"\nRoom {next_room} is already clean.")
    print("\nFinal Environment State:")
    print(f"Room A: {environment['A']}")
    print(f"Room B: {environment['B']}")
    print("All Rooms are now clean!")
     vacuum_agent()
```

OUTPUT:

```
vacuum_agent()

Current status of Room A (Dirty/Clean):  Clean
Current status of Room B (Dirty/Clean):  Dirty
Enter initial location of vacuum cleaner (A/B):  B

Initial Environment State:
Room A: clean
Room B: dirty
Vacuum Cleaner at Room: B

Room B is dirty. Cleaning
RoomB is cleaned.

Room A is already clean.

Final Environment State:
Room A: clean
Room B: dirty
All Rooms are now clean!
```

Program 5: Goal based agent.

Write a Python Program to simulate an Autonomous Car Agent .

```
def autonomous_car_agent(traffic_light, obstacle, road_condition):
    print("Traffic Light:", traffic_light)
    print("Obstacle Detected:", obstacle)
    print("Road Condition:", road_condition)
    if traffic_light.lower() == "red":
        action = "Stop the car"
    elif traffic_light.lower() == "yellow":
        action = "Slow down and prepare to stop"
    elif obstacle.lower() == "yes":
        action = "Stop — obstacle ahead"
    elif road_condition.lower() == "slippery":
        action = "Drive slowly and carefully"
    elif traffic_light.lower() == "green" and obstacle.lower() == "no":
        action = "Move forward safely"
    else:
        action = "Stay alert and maintain current state"
print("Decision:", action)
print(" Scenario 1 ")
autonomous_car_agent("Red", "No", "Clear")
print("\n Scenario 2 ")
autonomous_car_agent("Green", "Yes", "Clear")
print("\n  Scenario 3 ")
autonomous_car_agent("Green", "No", "Slippery")
```

OUTPUT:

```
 Scenario 1
Traffic Light: Red
Obstacle Detected: No
Road Condition: Clear
Decision: Stop the car

 Scenario 2
Traffic Light: Green
Obstacle Detected: Yes
Road Condition: Clear
Decision: Stop — obstacle ahead

  Scenario 3
Traffic Light: Green
Obstacle Detected: No
Road Condition: Slippery
Decision: Drive slowly and carefully
```

Program 6: Write a Python program that accepts an adjacency list of a graph as input and performs Breadth First Search (BFS) starting from a given source node.

```
n = int(input("Enter the number of nodes: "))
nodes = []
print("Enter the node names:")
for i in range(n):
    name = input(f"Node {i+1} name: ")
    nodes.append(name)
graph = {}
print("\nEnter the adjacent nodes for each node (separated by space or
comma):")
for node in nodes:
    raw = input(f"Neighbors of {node}: ")
    raw = raw.replace(",", " ")
    neighbors = raw.split()
    graph[node] = neighbors
print("\nAdjacency List Representation:")
for node in graph:
    print(node, "->", ", ".join(graph[node]))
start = input("\nEnter the starting node for BFS: ")
if start not in graph:
    print("\nError: Starting node not found in graph!")
    exit()
visited = []
queue = []
visited.append(start)
queue.append(start)
print("\nBFS Traversal Order:", end=" ")
while queue:
    node = queue.pop(0)
    print(node, end=" ")
    for neighbor in graph[node]:
        if neighbor not in visited:
            visited.append(neighbor)
            queue.append(neighbor)
print()
```

OUTPUT:

```
Enter the number of nodes:  3
Enter the node names:
Node 1 name:  A
Node 2 name:  B
Node 3 name:  C

Enter the adjacent nodes for each node (separated by space or comma):
Neighbors of A:  B,C
Neighbors of B:  A,C
Neighbors of C:  B

Adjacency List Representation:
A -> B, C
B -> A, C
C -> B

Enter the starting node for BFS:  A

BFS Traversal Order: A B C
```

Program 7: Write a Python program that accepts an adjacency list of a graph as input and performs Depth First Search (DFS) starting from a given source node.

```
n = int(input("Enter the number of nodes: "))
nodes = []
print("Enter the node names:")
for i in range(n):
    name = input(f"Node {i+1} name: ")
    nodes.append(name)
graph = {}
print("\nEnter the adjacent nodes for each node (separated by space):")
for node in nodes:
    neighbors = input(f"Neighbors of {node}: ").replace(",", " ").split()
    graph[node] = neighbors
print("\nAdjacency List Representation:")
for node in graph:
    print(node, "->", ", ".join(graph[node]))
start = input("\nEnter the starting node for DFS: ")
visited = []
stack = [start]
print("\nDFS Traversal Order:", end=" ")
while stack:
    node = stack.pop()
    if node not in visited:
        print(node, end=" ")
        visited.append(node)
        for neighbor in reversed(graph[node]):
            if neighbor not in visited:
                stack.append(neighbor)

print()
```

OUTPUT:

```
Enter the number of nodes:  4
Enter the node names:
Node 1 name:  a
Node 2 name:  b
Node 3 name:  c
Node 4 name:  d

Enter the adjacent nodes for each node (separated by space):
Neighbors of a:  b,c,d
Neighbors of b:  a,c,d
Neighbors of c:  a,b,d
Neighbors of d:  a,b,c

Adjacency List Representation:
a -> b, c, d
b -> a, c, d
c -> a, b, d
d -> a, b, c

Enter the starting node for DFS:  a

DFS Traversal Order: a b c d
```

Program 8: Write a Python program to evaluate basic Propositional Logic expressions. The user should enter truth values (True/False) for propositions P and Q.

```
print("enter the truth values for P and Q (true/false):")
P = input("enter P (true/false): ")
Q = input("enter Q (true/false): ")
if P.lower() == "true":
    P = True
else:
    P = False

if Q.lower() == "true":
    Q = True
else:
    Q = False
and_result = P and Q
or_result = P or Q
not_result = not P
implication_result = (not P) or Q
biconditional_result = (P and Q) or ((not P) and (not Q))

print("\n--- results ---")
print("P AND Q =", and_result)
print("P OR Q  =", or_result)
print("NOT P   =", not_result)
print("P -> Q  =", implication_result)
print("P <-> Q =", biconditional_result)
```

<u>OUTPUT:</u>

```
enter the truth values for P and Q (true/false):
enter P (true/false):  true
enter Q (true/false):  false

--- results ---
P AND Q = False
P OR Q  = True
NOT P   = False
P -> Q  = False
P <-> Q = False
```

Program 9: Write a Python Program that stores a few predicate facts (like parent, brother, teacher) using simple lists.

```
parent_facts = [("John","Marry"), ("Ravi","Amit")]
brother_facts = [("Amit","Rohan"), ("Rohan","Amit")]
teacher_facts = [("Seema","Class5"), ("Arun","Class3")]
print("Enter a Query like : Parent(John, Mary)")
query = input("Query: ")
parts = query.split("(")
predicate = parts[0]
args = parts[1].replace(")","")
arg1, arg2 =args.split(",")
arg1 = arg1.strip()
arg2 = arg2.strip()
fact_exists = False
if predicate == "Parent":
    if (arg1, arg2) in parent_facts:
        fact_exists = True
elif predicate == "Brother":
    if (arg1, arg2) in brother_facts:
        fact_exists = True
elif predicate == "Teacher":
    if (arg1, arg2) in teacher_facts:
        fact_exists = True
if fact_exists:
    print("Fact exists in the knowledge base.")
else:
    print("Fact does NOT exist in the knowledge base. ")
```

OUTPUT:

```
Enter a Query like : Parent(John, Mary)
Query:  teacher(Seema,Class5)
Fact does NOT exist in the knowledge base.
```

Program 10: Find Mean, Mode, Median and Standard Deviation of a given dataset speed=[99,86,87,88,111,86,103,87,94,78,77,85,86].

```
import numpy
from scipy import stats
speed=[99,86,87,88,111,86,103,87,94,78,77,85,86]
x=numpy.mean(speed)
y=numpy.median(speed)
z=stats.mode(speed)
p=numpy.std(speed)
print("Mean is",x)
print("Median is",x)
print("Mode is",x)
print("Standard Deviation is",x)
```

OUTPUT:

```
Mean is 89.76923076923077
Median is 89.76923076923077
Mode is 89.76923076923077
Standard Deviation is 89.76923076923077
```

Program 11: Write a Python Program to demonstrate the concept of a dataset and features . Create a small dataset, identify the features and target values ,and display them separately.

```
import numpy as np
X=np.array({1,2,3,4})
y=np.array({10,20,30,40})
print("Features before reshape(1-D):")
print(X)
print("Shape:",X.shape)
X=x.reshape(-1,1)
print("/nFeature aftert reshape (2-D):")
print(X)
print("Shape:",X.shape)
y= y.reshape(-1,1)
print("\nLabel after reshape(2-D):")
print(y)
print("Shape:",y.shape)
```

<u>OUTPUT:</u>

```
Features before reshape(1-D):
{1, 2, 3, 4}
Shape: ()
/nFeature aftert reshape (2-D):
[[89.76923077]]
Shape: (1, 1)

Label after reshape(2-D):
[[{40, 10, 20, 30}]]
Shape: (1, 1)
```

Program 12: Design a Python program to demonstrate the use of Simple imputer class for filling the missing values.

MEDIAN:

```
import numpy as np
from sklearn.impute import SimpleImputer
data = np.array([
    [10, 2],
    [15, np.nan],
    [np.nan, 4],
    [20, 5]
])
print("Original Data:")
print(data)
imputer = SimpleImputer(strategy='median')
imputed_data = imputer.fit_transform(data)
print("\nData After Imputation:")
print(imputed_data)
```

OUTPUT:

```
Original Data:
[[10.  2.]
 [15. nan]
 [nan  4.]
 [20.  5.]]

Data After Imputation:
[[10.  2.]
 [15.  4.]
 [15.  4.]
 [20.  5.]]
```

MOST FREQUENT:

```
import numpy as np
from sklearn.impute import SimpleImputer
from scipy import stats
data = np.array([
    [10, 2],
    [15, np.nan],
    [np.nan, 4],
    [20, 5]
])
print("Original Data:")
print(data)
imputer = SimpleImputer(strategy='most_frequent')
imputed_data = imputer.fit_transform(data)
print("\nData After Imputation:")
print(imputed_data)
```

OUTPUT:

```
Original Data:
[[10.   2.]
 [15.  nan]
 [nan   4.]
 [20.   5.]]

Data After Imputation:
[[10.   2.]
 [15.   2.]
 [10.   4.]
 [20.   5.]]
```

MEAN:

```
import numpy as np
from sklearn.impute import SimpleImputer
from scipy import stats
data = np.array([
    [10, 2],
    [15, np.nan],
    [np.nan, 4],
    [20, 5]
])
print("Original Data:")
print(data)
imputer = SimpleImputer(strategy='mean')
imputed_data = imputer.fit_transform(data)
print("\nData After Imputation:")
print(imputed_data)
```

OUTPUT:

```
Original Data:
[[10.   2.]
 [15.  nan]
 [nan   4.]
 [20.   5.]]

Data After Imputation:
[[10.          2.        ]
 [15.          3.66666667]
 [15.          4.        ]
 [20.          5.        ]]
```

Program 13: Write a Python program to find outliers using Z-score.

```
import numpy as np
from scipy import stats
data = [10, 12, 14, 15, 16, 18, 20, 22, 24, 100]
z_scores = stats.zscore(data)
threshold = 3
outliers = []
for i in range(len(data)):
   if abs(z_scores[i]) > threshold:
     outliers.append(data[i])
print("Data:", data)
print("Z-scores:", z_scores)
print("Outliers:", outliers)
```

<u>OUTPUT:</u>

```
Data: [10, 12, 14, 15, 16, 18, 20, 22, 24, 100]
Z-scores: [-0.59665155 -0.51762485 -0.43859816 -0.39908481 -0.35957146 -0.28054477
 -0.20151807 -0.12249138 -0.04346468  2.95954972]
Outliers: []
```

Program 14:Design a Python program to demonstrate how a dataset is divided into training data and testing data using the train_test_split() function from the scikit learn library.

```
import numpy as np
from sklearn.model_selection import train_test_split
x=np.array([800,1000,1200,1500,1800]).reshape(-1,1)
y=np.array([120,150,180,210,250])
x_train,x_test,y_train,y_test=train_test_split(
   x,y,test_size=0.2,random_state=42)
print("x_train:",x_train)
print("y_train:",y_train)
print("x_test:",x_test)
print("y_test:",y_test)
```

OUTPUT:

```
x_train: [[1800]
 [1200]
 [ 800]
 [1500]]
y_train: [250 180 120 210]
x_test: [[1000]]
y_test: [150]
```

Program 15:Given the following dataset representing marks scored by students:[10,12,14,15,16,18,20,22,24]. Write a Python program to perform y Min-Max Normalization to scale the data between 0 and 1.(min_max)

```
import numpy as np
data = np.array([10, 12, 14, 15, 16, 18, 20, 22, 24])
data_min = np.min(data)
data_max = np.max(data)
normalized_data = (data - data_min) / (data_max - data_min)
print("Original Data:", data)
print("Minimum Value:", data_min)
print("Maximum Value:", data_max)
print("Normalized Data:", normalized_data)
```

OUTPUT:

```
Original Data: [10 12 14 15 16 18 20 22 24]
Minimum Value: 10
Maximum Value: 24
Normalized Data: [0.         0.14285714 0.28571429 0.35714286 0.42857143 0.57142857
 0.71428571 0.85714286 1.        ]
```

Program 16: This program is used to implement Simple Linear Regression using Python and scikit-learn to predict house prices based on house size.

```
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
# Step 1: Create dataset
X = np.array([800, 1000, 1200, 1500, 1800]).reshape(-1, 1)
y = np.array([120, 150, 180, 210, 250])
# Step 2: Split data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
# Step 3: Create Linear Regression model
model = LinearRegression()
# Step 4: Train the model
model.fit(X_train, y_train)
# Step 5: Predict on test data
y_pred = model.predict(X_test)
y_pred1 = model.predict([[300]])
# Step 6: Display results
print("Test Data (X_test):", X_test)
print("Actual Price (y_test):", y_test)
print("Predicted Price (y_pred):", y_pred)
# Step 7: Display model parameters
print("Slope (Coefficient):", model.coef_)
print("Intercept:", model.intercept_)
print("Price value for area 300 sqft:", y_pred1)
```

OUTPUT:

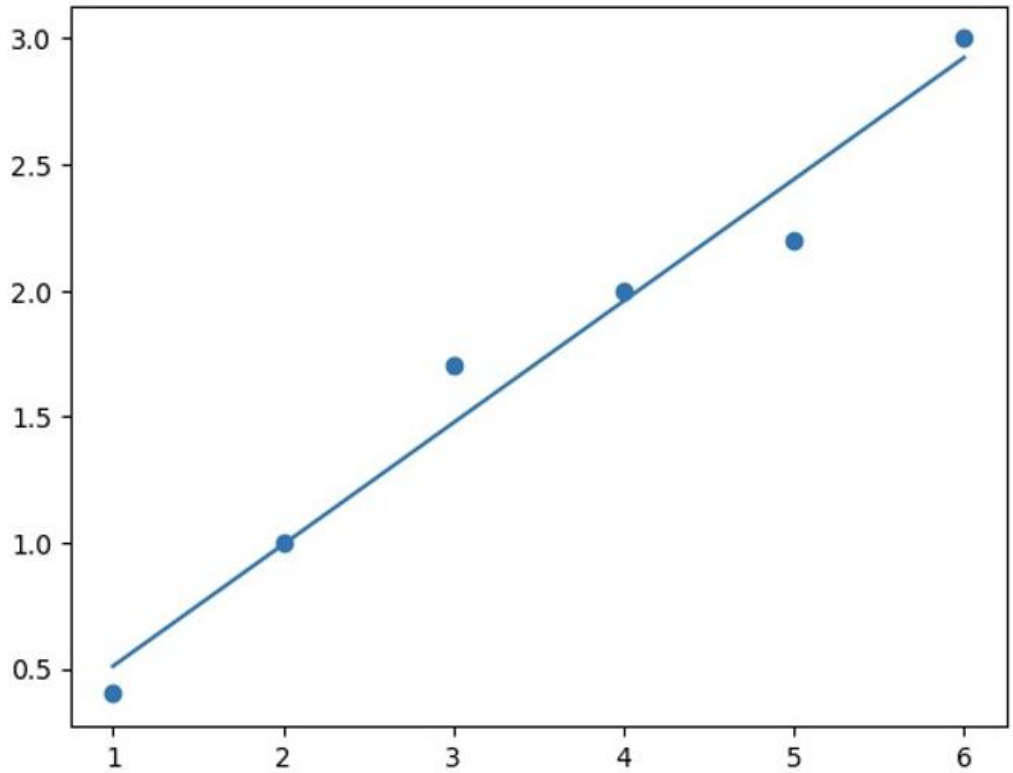
```
Test Data (X_test): [[1000]]
Actual Price (y_test): [150]
Predicted Price (y_pred): [148.44748858]
Slope (Coefficient): [0.12785388]
Intercept: 20.59360730593616
Price value for area 300 sqft: [58.94977169]
```

Program 17: This program is used to draw the best-fit regression line for a given dataset using linear regression.

```
import numpy as np
import matplotlib.pyplot as plt
from scipy import stats
x = [1, 2, 3, 4, 5, 6]
y = [0.4, 1.0, 1.7, 2.0, 2.2, 3.0]
slope, intercept, r, p, std_err = stats.linregress(x, y)
print(slope)
print(intercept)
def myfunc(x):
   return slope * x + intercept
mymodel = list(map(myfunc, x))
plt.scatter(x, y)
plt.plot(x, mymodel)
plt.show()
```

OUTPUT:

```
0.4828571428571428
0.02666666666666684
```



Program 18: Design and develop a python program that uses a decision tree classifier to predict diabetes outcomes.

```
import pandas as pd
from sklearn.tree import DecisionTreeClassifier, export_text
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
from sklearn import metrics
from pandas import DataFrame
pima = pd.read_csv("C:/Users/student/Desktop/diabetes.csv")
print("Dataset Preview:")
print(pima.head())
x = list(pima.columns)
x = pima.drop("Outcome",axis = 1)
y = pima["Outcome"]
print("\nPredictor Attributes : ")
print(x.columns.tolist())
x_train,x_test,y_train,y_test = train_test_split(x,y,test_size = 0.3,random_state = 1)
clf = DecisionTreeClassifier()
clf = clf.fit(x,y)
y_pred = clf.predict(x_test)
accuracy = accuracy_score(y_test,y_pred)
print("\nModel Accuracy : ",accuracy)
```

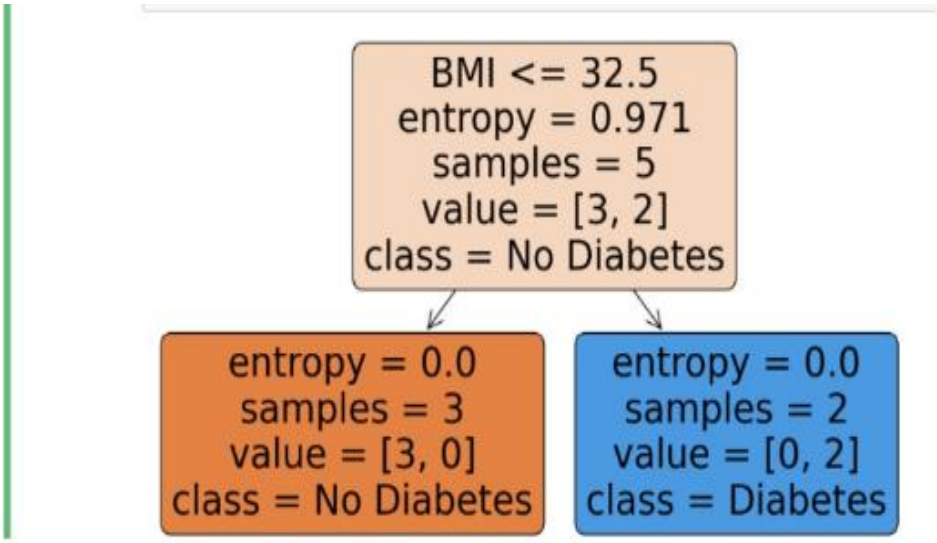
OUTPUT:

```
Dataset Preview:
   Pregnancies  Glucose  BloodPressure  SkinThickness  Insulin   BMI  \
0            6      148             72             35        0  33.6
1            1       85             66             29        0  26.6
2            8      183             64              0        0  23.3
3            1       89             66             23       94  28.1
4            0      137             40             35      168  43.1

   DiabetesPedigreeFunction  Age  Outcome
0                     0.627   50        1
1                     0.351   31        0
2                     0.672   32        1
3                     0.167   21        0
4                     2.288   33        1

Predictor Attributes :
['Pregnancies', 'Glucose', 'BloodPressure', 'SkinThickness', 'Insulin', 'BMI', 'DiabetesPedigreeFunction', 'Age']

Model Accuracy :  1.0
```

Model Accuracy: 1.0

```
from sklearn.tree import DecisionTreeClassifier, plot_tree
import matplotlib.pyplot as plt
# Step 1: Dataset
# Features: [Glucose, BMI]
X = [
    [120, 30],
    [85, 25],
    [90, 28],
    [160, 35],
    [200, 40]
]
# Labels: 0 = No Diabetes, 1 = Diabetes
y = [0, 0, 0, 1, 1]
# Step 2: Create Decision Tree model using ENTROPY
model = DecisionTreeClassifier(criterion="entropy")
model.fit(X, y)
# Step 3: Visualize the tree
plt.figure(figsize=(10, 6))
plot_tree(
    model,
    feature_names=["Glucose", "BMI"],
    class_names=["No Diabetes", "Diabetes"],
    filled=True,
    rounded=True
)
plt.show()
```

OUTPUT:



Program 19: Write a python program to implement logistic regression for binary classification problem that predicts whether a student passes or fails based on the number of hours studied.

```
import numpy as np
from sklearn.linear_model import LogisticRegression
X = np.array([[1], [2], [3], [4], [5]])
y = np.array([0, 0, 0, 1, 1])
model = LogisticRegression()
model.fit(X, y)
hours = [[3.5]]
prediction = model.predict(hours)
probability = model.predict_proba(hours)
print("Hours Studied:", hours[0][0])
print("Predicted Class (0=Fail, 1=Pass):", prediction[0])
print("Probability [Fail, Pass]:", probability[0])
```

OUTPUT:

```
Hours Studied: 3.5
Predicted Class (0=Fail, 1=Pass): 0
Probability [Fail, Pass]: [0.5208689 0.4791311]
```

Program 20: Write a program to Implement algorithm for K-means.

```
import pandas as pd
from sklearn.cluster import KMeans
# Step 1: Load data from CSV file
data = pd.read_csv("C:/Users/student.AUBLRU/Desktop/data.csv")
# Step 2: Create KMeans model
model = KMeans(n_clusters=2, random_state=0)
# Step 3: Fit the model
model.fit(data)
# Step 4: Display cluster labels
print("Cluster labels:")
print(model.labels_)
# Optional: View cluster centers
print("\nCluster centers:")
print(model.cluster_centers_)
```

OUTPUT:

```
C:\ProgramData\anaconda3\Lib\site-packages\sklearn\cluster\_kmeans.py:1412:
ureWarning: The default value of `n_init` will change from 10 to 'auto' in
Set the value of `n_init` explicitly to suppress the warning
  super()._check_params_vs_input(X, default_n_init=10)
C:\ProgramData\anaconda3\Lib\site-packages\sklearn\cluster\_kmeans.py:1436:
rWarning: KMeans is known to have a memory leak on Windows with MKL, when t
are less chunks than available threads. You can avoid it by setting the env
ment variable OMP_NUM_THREADS=1.
  warnings.warn(

Cluster labels:
[1 1 1 0 0 0]

Cluster centers:
[[10.  2.]
 [ 1.  2.]]
```